So, while the original code performed a pointer-to-integer cast and then wrote back an integer (or, equivalently, a physical pointer with no provenance information), the changed program leaves the original provenance information intact. In other words, load-store elimination can result in increasing provenance restrictions on pointers, which can in turn result in introducing UB. To fix this, we let load return **poison** instead of performing an implicit cast.

The downside of disallowing implicit casts is that there is no longer a type that can actually hold any data that can be present in memory. LLVM typically uses integer types for that purpose but, as discussed above, that does not work in our semantics. One option to fix this is adding a new "data" type that only permits a few bitwise operations, and that can hence hold both pointer and integer bits. We intend to pursue this avenue further in the future.

Non-deterministic Values. We have defined some operations to yield a non-deterministic result, e.g., **icmp ule** of pointers of different blocks. This blocks duplication of instructions since they could then return different results. Solutions to this problem could include use of **undef** instead, which has its drawbacks [Lee et al. 2017], or explore the use of "entangled" instructions that need to be folded together. We leave this exploration for future work.

Out of memory. A technical problem with our memory model is that any transformation that may eliminate an out-of-memory condition is unsound. However, the necessary conditions for achieving miscompilation via this unsoundness are difficult to meet. First, the compiler must perform a transformation based on knowledge that a program path triggers out-of-memory (LLVM does not do this at present, and seems unlikely to do so in the future). Second, it would need to perform an optimization eliminating the out-of-memory condition (these transformations, such as eliminating unused allocations, are performed routinely). Designing a memory model that is sound in this respect seems challenging and it is not clear the result would be useful in practice.

9 CONCLUSION

Languages like C, C++, and Rust give the programmer low-level control over how memory is arranged and accessed while also giving the compiler freedom to perform high-level memory optimizations. This is challenging and current toolchains based on LLVM and GCC are not always able to strike the right balance between performance of generated code and strength of guarantees made to programs. Part of the problem is that the memory models for the internal representations in these compilers—where high-level optimizations are performed—are informally specified and thus easily misunderstood.

We created a new memory model for LLVM IR that we believe provides sufficiently strong guarantees that it can be efficiently targeted by front-ends while also permitting many desirable optimizations. We have implemented a prototype of this model and have shown that the implementation is not invasive, that code quality remains good, and that several known miscompilations due to the current, informal memory model are fixed.

ACKNOWLEDGMENTS

The authors thank Sanjoy Das, Alberto Magni, and Matthew Parkinson for feedback and discussions on previous versions of this paper. This work was supported in part by the Basic Science Research Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Science and ICT (2017R1A2B2007512), by an ERC Consolidator Grant for the project "RustBelt" (grant no. 683289), and by the National Science Foundation (grant no. 1218022). The first author was supported by a Korea Foundation for Advanced Studies scholarship.